UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS

SW505U CONSTRUCCIÓN DE SOFTWARE I

DOCENTE: Mag. Juan Luis Herencia Guerra

ENTREGABLE 1

Grupo

DIANA VALERI CHAVEZ PALOMINO 20240210D

IVAN ALEJANDRO NAVARRO PALACIOS 20231201F

ASTRID XIOMARA KAROLINE SALAZAR CARRANZA 20244743G

2026

1. Cronograma de Actividades y Entregables

Semana	Fase/Hito	Actividades Principales	Casos de Uso del Sistema
Semana 11	Planificación	Presentación de la estrategia de implementación, definición del stack tecnológico (Python, MySQL, JS) y configuración de entornos.	Ninguno
Semana 12	**Hito 1:** Núcleo del Dominio y Arquitectura Base.	Programar las entidades conceptuales puras (ADTs) del MTC y configurar la persistencia desacoplada (Patrón DAO).	Ninguno
	Hito 2: Módulo de Gestión de Empresas Transportistas.	Implementar el registro y actualización de datos de la empresa, validación de RUC y gestión del representante legal.	CU-01: Registrar empresa transportista. CU-02: Modificar datos de la empresa.
	Hito 3: Módulo de Gestión de Vehículos.	Implementar el registro de la flota vehicular, validación de placas y control de vigencias de SOAT y CITV.	CU-03: Registrar vehículo. CU-04: Modificar vehículo.
Semana 13	**Hito 4:** Módulo de Gestión de Conductores.	Implementar el registro de choferes, validación de DNI y control de categorías de licencias de conducir.	CU-05: Registrar conductor. CU-06: Modificar conductor.
	Hito 5: Módulo de Gestión de Solicitudes y Expedientes.	Desarrollar la creación de expedientes TUPA, lógica de adjuntos (PDFs) y visualización del estado del trámite.	CU-07: Presentar solicitud de autorización. CU-08: Presentar solicitud de renovación. CU-09: Presentar solicitud de modificación. CU-10: Presentar solicitud de incremento vehicular. CU-11: Consultar estado de solicitud.
	Hito 6: Módulo de Evaluación de Expedientes.	Configurar el flujo de revisión, registro de observaciones por	CU-12: Evaluar solicitud. CU-13: Registrar observación.

		inconsistencias normativas y recepción de subsanaciones.	CU-14: Presentar subsanación. CU-15: Evaluar subsanacion.
Semana 14	**Hito 7:** Módulo de Resoluciones y Autorizaciones.	Programar la emisión de decisiones finales, generación de autorizaciones y consultas de estado.	CU-16: Registrar resolución. CU-17: Consultar autorizaciones.
	Hito 8: Control de Vigencias y Fiscalización.	Programar la actualización automática a estado "Vencido", aplicar roles de seguridad.	(Procesos automáticos y Requerimientos No Funcionales)
	Hito 9: Pruebas unitarias y de integración.	Ejecución de pruebas unitarias y pruebas de integración para verificar la interacción entre módulos y persistencia. Corrección de errores y repetición de las pruebas afectadas.	Pruebas aprobadas y resultados documentados.

2. Descripción de Hitos de Desarrollo

2.1. Hito 1: Núcleo del Dominio y Arquitectura Base

En este hito se construye el corazón inmutable del sistema (la Capa de Entidades).

- Se programan los Tipos de Datos Abstractos (ADTs) puros, los cuales encapsulan las reglas normativas del MTC sin depender de frameworks ni almacenamiento.
- Se implementa el patrón Data Access Object (DAO) en la capa de infraestructura, garantizando que la lógica central en Python no tenga referencias directas al motor MySQL ni contenga sentencias SQL, respetando la Arquitectura Limpia.

2.2. Hito 2: Módulo de Gestión de Empresas Transportistas

Este hito se centra en establecer la identidad jurídica de los administrados que interactuarán con el MTC.

- Se programa el registro y actualización de datos fiscales, asegurando validaciones estrictas (ej. formato de RUC de 11 dígitos y unicidad en la base de datos).
- Se desarrolla la vinculación lógica de la cuenta de acceso (Usuario) con el rol de Representante Legal para acreditar su facultad de iniciar trámites.

2.3. Hito 3: Módulo de Gestión de Vehículos

Este hito se enfoca en el aspecto físico y técnico del parque automotor, asegurando que el sistema refleje la realidad operativa de la empresa.

- Se programa el registro de características técnicas y configuraciones vehiculares (placa, chasis, ejes, pesos), asegurando la unicidad de las placas a nivel nacional.
- Se integra directamente en la entidad vehicular la lógica para registrar y controlar las fechas de caducidad de certificaciones críticas como el SOAT y el CITV (Certificado de Inspección Técnica Vehicular).

2.4. Hito 4: Módulo de Gestión de Conductores

Este hito construye la nómina operativa de las empresas transportistas y valida su aptitud legal.

- Se desarrolla el registro, actualización y desvinculación de conductores.
- Se implementan las reglas de negocio para registrar las licencias de conducir, diferenciando categorías estándar y requerimientos especiales (como la categoría especial para Materiales Peligrosos - MatPel).

2.5. Hito 5: Módulo de Gestión de Solicitudes y Expedientes

Este hito marca el inicio de la transaccionalidad administrativa. Se orquesta el inicio de los trámites regulados por el TUPA.

- Se desarrolla la lógica de ramificación para atender distintos tipos de trámites: autorizaciones iniciales, renovaciones, modificaciones y procedimientos vehiculares (incremento/sustitución).
- Se implementa la persistencia escalable de documentos adjuntos (PDFs) al expediente digital y se habilita la trazabilidad para que la empresa consulte en tiempo real el avance de su gestión.

2.6. Hito 6: Módulo de Evaluación de Expedientes

En este hito se aísla el componente de control y revisión técnica a cargo del personal del MTC.

- Se configura un mediador lógico para que el Evaluador acceda al expediente sin alterar los datos originales de la empresa.
- Se programa el ciclo de observaciones: el registro de inconsistencias normativas, el cálculo de plazos de respuesta, el cambio automático de estado y la capacidad de la empresa para subir la subsanación correspondiente.

2.7. Hito 7: Módulo de Resoluciones y Autorizaciones

En este hito se materializan las decisiones administrativas y se generan los títulos habilitantes.

- Se desarrolla la capacidad del Responsable MTC para emitir resoluciones de aprobación o denegación.
- Se implementa la lógica que, tras una aprobación, genera de forma automática la Autorizacion (para la empresa), registrando el inicio y fin de su vigencia operativa.

2.8. Hito 8: Control de Vigencias y Fiscalización

Este hito se enfoca exclusivamente en la calidad, seguridad, interoperabilidad y cumplimiento normativo del sistema.

- Se programa un proceso automático (cron) que audita diariamente la base de datos para cambiar al estado "Vencido" aquellas autorizaciones, certificados y licencias que superen su plazo legal.
- Se implementa el Control de Acceso Basado en Roles (RBAC) con tokens JWT, protegiendo las rutas según el actor (Empresa, Evaluador, Responsable).

2.9. Hito 9: Estabilización Final

Es el hito de preparación para la entrega final y sustentación del proyecto.

- Se ejecutan pruebas unitarias sobre los casos de compatibilidad de categorías y pruebas integrales sobre el flujo completo de una solicitud.
- Se aplica la resolución de bugs detectados, optimización de consultas SQL en MySQL, limpieza general del código Python y ajustes finales solicitados por el docente.

3. Entorno de Desarrollo y Control de Versiones

Esta sección establece los lineamientos técnicos, operativos y las convenciones de construcción para el equipo de desarrollo del Sistema de Gestión de Autorizaciones de Transporte Terrestre (MTC). Su propósito es garantizar un ecosistema de trabajo uniforme fundamentado en los principios de minimización de la complejidad y anticipación al cambio, facilitando la integración progresiva de módulos y asegurando el cumplimiento de la Arquitectura Limpia.

3.1. Estrategia de Control de Versiones

El proyecto emplea Git como motor de control de versiones y GitHub como repositorio centralizado. A diferencia de modelos pesados, el equipo ha decidido adoptar la estrategia GitHub Flow.

3.1.1. Justificación de la Estrategia Seleccionada

Para un equipo de desarrollo ágil y de tamaño reducido, mantener múltiples ramas de larga duración (como dicta *GitFlow* con main y develop) genera una sobrecarga administrativa innecesaria. GitHub Flow es una alternativa más ligera y moderna orientada a la Integración Continua (CI). Se basa en una única regla fundamental: la rama principal (main) siempre debe ser estable y estar lista para producción. Todo desarrollo nuevo se hace en ramas de vida corta que, tras ser verificadas, se fusionan directamente a la principal.

3.1.2. Estructura de Ramas

El repositorio utilizará una estructura simplificada de dos niveles:

- **main (Rama Principal):** Contiene la versión oficial, estable y funcional del sistema. No se permiten *commits* directos bajo ninguna circunstancia.
- **feature/* (Ramas de Funcionalidad):** Ramas de vida corta creadas a partir de main para aislar el desarrollo de casos de uso específicos. Al finalizar, se eliminan.
  Ejemplos del proyecto:
  - feature/empresas
  - feature/vehiculos
  - feature/solicitudes
  - feature/evaluacion

3.1.3. Flujo de Trabajo y Revisión Técnica

El proceso colaborativo seguirá este ciclo disciplinado:

1. El desarrollador sincroniza su entorno local y crea una rama feature desde main.
2. Implementa la funcionalidad aplicando el diseño detallado y construye las pruebas unitarias correspondientes (*Construcción para la verificación*).
3. Publica la rama en GitHub y abre un *Pull Request (PR)*.

4. Una vez aprobada, la rama se fusiona con main y se elimina la rama feature.

3.1.4. Convención de Commits

Se empleará el estándar *Conventional Commits* para mantener un historial trazable y automatizable:

- **feat:** Nueva funcionalidad
- **fix:** Corrección de defectos
- **refactor:** Reestructuración interna sin alterar funcionalidad
- **docs:** Actualización de documentación técnica.
- **test:** implementación de pruebas

3.2. Estándares de Código

Para producir software legible, verificable y con un alto grado de mantenibilidad, el equipo aplicará rigurosamente los principios de *Clean Code* y el diseño orientado a la implementación.

3.2.1. Convenciones de Nombres (Nomenclatura Significativa)

- **Clases y ADTs (PascalCase):** Representan el núcleo del dominio. Ej: EmpresaTransportista, Vehiculo, ActaFiscalizacion.
- **Métodos y Funciones (camelCase):** Funciones de propósito único. Se prioriza el uso de verbos de acción. Ej: evaluarSolicitud(), registrarObservacion().
- **Variables (camelCase):** Nombres descriptivos y explícitos. Ej: fechaVencimientoSoat, cantidadEjes.
- **Constantes (UPPER_SNAKE_CASE):** Variables inmutables y reglas parametrizables. Ej: PLAZO_SUBSANACION_DIAS, MAX_HORAS_CONDUCCION.

3.2.2. Principios de Diseño

- **Principios SOLID:**
  - *Responsabilidad Única (SRP):* Separación estricta de responsabilidades. Un controlador web no debe contener lógica de validación de placas ni sentencias SQL; esa es tarea exclusiva de la capa de Casos de Uso y Persistencia, respectivamente.
  - *Abierto/Cerrado (OCP):* El sistema está diseñado para incorporar nuevas normativas (ej. nuevas modalidades de carga MatPel) mediante extensión, sin necesidad de alterar la lógica central existente.
- **Principio DRY (Don't Repeat Yourself):** Las reglas de negocio, como la validación de un formato de placa o un RUC, se programarán como Objetos de Valor (*Value Objects*) en el núcleo de la aplicación, reutilizándose en todos los módulos sin duplicar código.

3.2.3. Manejo Disciplinado de Errores

Se erradicarán los bloques catch vacíos. El sistema capturará fallos en los límites de los componentes y lanzará excepciones de dominio explícitas (ej. VehiculoNoAptoException o LicenciaIncompatibleException). Los errores técnicos (como caídas de base de datos) se registrarán en bitácoras (logs) devolviendo mensajes genéricos al usuario por seguridad.

3.3. Estructura de Carpetas Basada en Arquitectura Limpia

El proyecto separa de forma absoluta el backend transaccional del frontend interactivo, garantizando la independencia tecnológica y el bajo acoplamiento. La siguiente estructura refleja la división por módulos del MTC y el respeto estricto a las capas de la Arquitectura Limpia.

```
sistema_autorizaciones_mtc/
├── backend/
│   ├── src/
│   │   ├── application/              # Casos de uso (Orquestación del sistema)
│   │   │   ├── empresas/
│   │   │   │   ├── dtos/
│   │   │   │   └── use_cases/
│   │   │   ├── vehiculos/
│   │   │   ├── solicitudes/
│   │   │   ├── evaluaciones/
│   │   │   └── resoluciones/
│   │   ├── domain/                   # Núcleo del MTC (Reglas y ADTs inmutables)
│   │   │   ├── entities/             # EmpresaTransportista, Vehiculo, Autorizacion
│   │   │   ├── enums/                # TipoTramite, CategoriaVehicular, EstadoAutorizacion
│   │   │   ├── exceptions/           # Excepciones de dominio
│   │   │   ├── repositories/         # Interfaces puras para persistencia (Contratos)
│   │   │   └── value_objects/        # Ruc, Placa, Brevete
│   │   ├── infrastructure/           # Adaptadores externos y frameworks
│   │   │   ├── api/                  # Controladores web (Rutas de FastAPI)
│   │   │   ├── database/             # Configuración del motor MySQL
│   │   │   └── persistence/          # Implementación concreta de los Repositorios (DAO)
│   │   ├── shared/
│   │   │   ├── config/
│   │   │   └── utils/
│   │   └── main.py                   # Punto de entrada de la aplicación
│   ├── tests/
│   │   ├── integration/              # Pruebas integrales de flujo completo TUPA
│   │   └── unit/                     # Pruebas unitarias aisladas por módulo
│   ├── requirements.txt
│   └── .env
└── frontend/
    ├── public/
    │   ├── assets/                   # Imágenes, íconos y logotipos del MTC
    │   └── css/                      # Hojas de estilo y diseño responsivo
    └── src/
```

```
│   ├── views/                  # Archivos HTML (bandejas, formularios)
│   │   ├── evaluador_mtc/
│   │   └── representante_empresa/
│   ├── js/
│   │   ├── components/         # Lógica visual y manipulación del DOM
│   │   ├── services/           # Clientes Fetch API para conexión con el backend
│   │   └── utils/              # Formateadores (fechas, RUC, validaciones frontend)
└── README.md
```

Justificación Arquitectónica: Esta disposición aísla completamente las dependencias tecnológicas. El frontend (Vanilla JS/HTML) ignora cómo está construido el backend; a su vez, las reglas de negocio del MTC en domain y application (Python) ignoran por completo la existencia de una base de datos MySQL, interactuando únicamente a través de puertos y adaptadores definidos en infrastructure.

3.4. Entorno de Desarrollo

Para garantizar la homogeneidad en la construcción y mitigar errores de despliegue originados por discrepancias en las herramientas ("en mi máquina sí funciona"), se establece un ecosistema tecnológico estandarizado para todo el equipo:

Componente	**Herramienta / Tecnología**	**Propósito en el Proyecto**
Lenguaje Backend	Python 3.12+	Desarrollo de la lógica central y algoritmos del negocio.
Framework Backend	FastAPI	Exposición de endpoints RESTful eficientes.
Gestor de Paquetes	pip	Instalación y administración de dependencias de Python.
Frontend	HTML5, CSS3, Vanilla JS	Desarrollo de interfaces responsivas y ligeras sin compiladores.
Motor de Base de Datos	MySQL 8.0+	Persistencia relacional de expedientes, flotas y resoluciones.
Administrador DB	MySQL Workbench	Interfaz gráfica para diseño físico y consultas directas (SQL).
Control de Versiones	Git / GitHub	Aislamiento de ramas (GitHub Flow) y revisión de código.

Componente	Herramienta / Tecnología	Propósito en el Proyecto
Entorno de Desarrollo	Visual Studio Code	IDE principal con soporte para análisis estático básico (Linters).
Cliente HTTP	Postman	Ejecución de pruebas aisladas a la API del backend.

3.4.1. Comandos de Inicialización del Entorno

- **Configuración del Backend (Python):** El equipo utilizará entornos virtuales para evitar colisiones de librerías globales. Desde la terminal en la raíz del backend:

```
# Crear el entorno virtual
python -m venv venv
# Activar el entorno (Windows)
venv\Scripts\activate
# Instalar dependencias del proyecto
pip install -r requirements.txt
# Levantar el servidor de desarrollo
fastapi dev main.py
```

2. Configuración del Frontend (HTML/JS): Al emplear JS y HTML puro, el frontend no requiere procesos complejos de compilación. Para ejecutar el entorno visual, bastará con utilizar un servidor de desarrollo estático ligero que servirá la carpeta frontend/src/views/ en el puerto local y aplicará *Hot Reload* automático ante cualquier modificación del código.

4.GUÍA DE DESPLIEGUE E INFRAESTRUCTURA

Esta sección detalla la configuración física y lógica del entorno donde operará el Sistema de Gestión de Autorizaciones de Transporte Terrestre, garantizando la inalterabilidad de los expedientes y la alta disponibilidad del servicio.

4.1. Arquitectura de servidor y alojamiento

El sistema emplea una arquitectura física de tres componentes principales operando en la red de retorno local, los cuales actúan como los contenedores físicos de las capas lógicas definidas por la Arquitectura Limpia.

- **Capa de Presentación Física (Frontend - HTML/CSS/JS):** Corresponde a la capa más externa de la Clean Architecture. Es una interfaz a nivel de negocio responsiva: se encarga exclusivamente de renderizar las vistas del sistema y enviar peticiones HTTP.
- **Capa de Aplicación y Dominio (Backend - Python):** Funciona como el servidor central y es el contenedor del núcleo de la Arquitectura Limpia. Dentro de este servidor residen las capas internas inmutables:
    - **Capa de Casos de Uso:** Donde se orquestan las validaciones de negocio, como la compatibilidad de licencias y categorías vehiculares.
    - **Capa de Entidades**: Donde viven los Tipos de Datos Abstractos (ADTs) puros como Autorizacion, Vehiculo y SolicitudAdministrativa.
    - **Controladores/Adaptadores:** Reciben las peticiones del frontend en JS y las traducen para el núcleo.
- **Capa de Persistencia y Datos (MySQL):** Representa el detalle externo en la Clean Architecture. Opera como un servicio independiente en el sistema operativo. El backend de Python interactúa con esta base de datos de manera completamente transparente a través de puertos, garantizando que el núcleo desconozca cómo MySQL guarda los datos físicamente.

4.2 Instalación y configuración de MySQL en producción

Para asegurar que el sistema soporte el almacenamiento escalable y cumpla con las métricas de rendimiento, el motor de base de datos se aprovisionara y configurará bajo los siguientes parámetros técnicos:

4.2.1. Aprovisionamiento y Sintonización del Servicio

- **Instalación Base:** Se requiere la instalación nativa del motor MySQL 8.0+ en el sistema operativo del equipo anfitrión. El motor operará como un servicio en segundo plano.
- **Asignación de Memoria:** Se ajustará el parámetro innodb_buffer_pool_size para asignar al menos 512 MB de la memoria RAM del equipo exclusivamente al

almacenamiento en caché de MySQL. Esto asegurará que las consultas de expedientes e historiales se ejecuten rápidamente bajo concurrencia.

4.2.2. Materialización del Modelo Físico (Migraciones DDL y DML) Para que el backend de Python pueda operar, la estructura de la base de datos bd_mtc será inyectada mediante dos scripts secuenciales ejecutados desde la consola de MySQL Workbench:

- **Script DDL (Data Definition Language - Estructura):** Materializa el Modelo Físico de Datos, creando las tablas principales (EMPRESA_TRANSPORTISTA, VEHICULO, SOLICITUD_ADMINISTRATIVA, AUTORIZACION, RESOLUCION).
    - *Validaciones físicas:* El script DDL incluirá restricciones CHECK a nivel de motor. Por ejemplo, garantizará físicamente que el campo estado_autorizacion solo acepte 'Vigente', 'Suspendida', 'Vencida' o 'Cancelada', añadiendo una capa extra de integridad referencial.
- **Script DML (Data Manipulation Language - Datos Semilla):** Insertará un volumen representativo de Empresas Transportistas, flotas de vehículos y usuarios del MTC.
    - *Casos límite:* El script incluirá autorizaciones con fecha de vencimiento pasada, lo que permitirá la actualización automática a estado "Vencido" dictada por los casos de uso.

4.2.3. Aislamiento de Seguridad y Rol de Conexión

- **Principio de Mínimo Privilegio:** Queda estrictamente prohibido que el servidor de Python se conecte a la base de datos utilizando el usuario root.
- **Creaciones de roles:** Se crearán en MySQL tres usuarios específicos alineados con los actores (RNF-SEG-01):
    - rol_empresa_transportista: Permisos de SELECT, INSERT y UPDATE restringidos a las tablas de sus propias solicitudes y vehículos.
    - rol_personal_mtc: Permisos de lectura total de expedientes y permisos de escritura en la tabla OBSERVACION. Sin permisos de DDL.
    - rol_SUTRAN: Permisos para registrar actas de fiscalización y consultar las actas remitidas por su institución. La evaluación y los cambios de estado de las autorizaciones corresponden al personal autorizado del MTC.

5.GUÍA DE OPERACIONES Y MANTENIMIENTO

5.1. Gestión de Servicios

Para garantizar que la aplicación y la base de datos se ejecuten de forma ininterrumpida y se inicien automáticamente ante reinicios inesperados del servidor, se utilizará el gestor de servicios nativo de Linux (systemd).

Motor de Base de Datos (MySQL):

- Verificar estado: sudo systemctl status mysql
- Iniciar servicio: sudo systemctl start mysql

- Detener servicio: sudo systemctl stop mysql
- Reinicio seguro: sudo systemctl restart mysql

Servidor del Backend (API de Python):

- Verificar estado: sudo systemctl status mtc-api
- Iniciar servicio: sudo systemctl start mtc-api
- Detener servicio: sudo systemctl stop mtc-api
- Reinicio: sudo systemctl restart mtc-api

5.2. Gestión de Variables de entorno

Siguiendo las buenas prácticas internacionales de desarrollo seguro, ninguna credencial de acceso o cadena de conexión a la base de datos estará incrustada de manera estática en el código fuente de Python.

Toda la configuración sensible se administrará mediante un archivo oculto .env exclusivo del servidor de producción.

Estructura del archivo de configuración .env en producción:

```
# Configuracion general del servidor
PORT=8000
PYTHONENV=production

# Parametros de conexion a MySQL
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=bd_mtc
DB_USER=rol_admin_mtc
DB_PASSWORD=[Contraseña_Compleja_Generada_En_Servidor]

# Seguridad de Tokens, Auditoria y Contraseñas
JWT_SECRET=[Firma_Algoritmica_Larga_Para_Sesiones_RBAC]
JWT_EXPIRATION=12h
```

5.3. Plan de Respaldos Automatizados

Para mitigar el riesgo de pérdida de información histórica y transaccional crítica (preservación de datos por 5 años exigida en RNF-NOR-01), se establece una política de copias de seguridad automáticas tanto de la base de datos.

El sistema programará una tarea diaria a través del servicio cron de Linux, a ejecutarse en horario de nula actividad transaccional (02:00 a.m.). La herramienta utilizada será mysqldump.

5.4. Plan de Recuperación Ante Desastres

En caso de presentarse una caída crítica del sistema o corrupción en los expedientes, el operador técnico debe aplicar el siguiente protocolo de contingencia:

- Fase 1 - Contención y Aislamiento: Detener temporalmente la API de Python para congelar el tráfico entrante de solicitudes y evitar la escritura de datos corruptos:sudo systemctl stop mtc-api
- Fase 2 - Restauración de Datos y Archivos: Si la base de datos presenta corrupción, se debe limpiar el entorno y restaurar el último respaldo diario válido (.sql) usando la consola de MySQL.
- Fase 3 - Levantamiento: Reiniciar el motor de la base de datos MySQL y reactivar el backend en Python:sudo systemctl restart mysqlsudo systemctl start mtc-api

Finalmente, el empleado del MTC ingresará a la interfaz web para consultar el estado de una solicitud administrativa existente. La respuesta exitosa y la correcta visualización del PDF adjunto determinarán formalmente el cierre de la contingencia y el retorno a la operatividad.